

Backend Dev Interview Prep Guide

For Kavya Patil • Study in this exact order

★ HIGH PRIORITY ♦ MEDIUM ◇ GOOD TO KNOW

PHASE 1 — Your Projects (Do This First)

Everything else builds on your ability to narrate your own work. Prepare these before touching any concept.

1. Project Pitches ★ HIGH PRIORITY

- Social Spending Platform — 90 sec pitch (why → what → challenge → improvement)
- Digital Dockets — 90 sec pitch, focus on YOUR specific role in the team
- House Price Predictor — 60 sec pitch
- For each: 1 technical challenge you faced + how you solved it
- For each: 1 thing you'd do differently with more time
- DSA used — name any data structure or algorithm applied (e.g. hash maps for O(1) lookup, sorting for ranking expenses)
- OOP used — which principles did you apply and where (e.g. encapsulation in model classes, inheritance for user roles, composition for strategies)
- How scalable — explain one bottleneck and how you'd address it (e.g. add caching, DB indexing, horizontal scaling, async processing)
- Situational questions — prepare answers for: What would you do if the API went down in prod? How would you handle a teammate missing a deadline? What if requirements changed mid-sprint?

💡 Write these out, then speak them aloud. Recording yourself and listening back is the single highest ROI activity.

PHASE 2 — Core Backend (Study Days 1–3)

2. HTTP & REST API Design ★ HIGH PRIORITY

- HTTP methods: GET, POST, PUT, PATCH, DELETE — when to use each
- Status codes: 200, 201, 204, 400, 401, 403, 404, 409, 500 — know what each means
- Request lifecycle: client → DNS → server → handler → response
- Idempotency — what it means and which methods are idempotent
- REST principles: stateless, resource-based URLs, consistent naming
- Versioning: /v1/ in path vs headers
- Pagination: offset vs cursor-based
- Error response design: consistent JSON error objects

💡 Anchor to your FastAPI project: 'In the Social Spending Platform, I designed endpoints like POST /expenses with Pydantic validation...' This is a stronger answer than any textbook definition.

3. Authentication & JWT ★ HIGH PRIORITY

- JWT structure: header.payload.signature — what goes in each part
- How signing works: server signs with secret, verifies on each request
- Access tokens vs refresh tokens — why two tokens?
- Token storage: localStorage (XSS risk) vs httpOnly cookie (CSRF risk) — tradeoffs
- Token expiry and revocation strategies
- RBAC (Role-Based Access Control) — roles as claims in JWT payload

- OAuth2 flow overview: authorization code, client credentials (just vocabulary)

💡 Tie directly to Digital Dockets: 'We needed role-based access for different government departments, so I implemented JWT where each token carries a role claim checked by middleware.'

4. Middleware ★ HIGH PRIORITY

- What middleware is: functions that intercept request/response in a pipeline
- Cross-cutting concerns: auth, logging, rate limiting, CORS — handled in middleware, not routes
- Auth middleware: extract JWT from header → verify → attach user to request
- Logging middleware: log method, path, status code, response time
- CORS middleware: what CORS is, why it exists, how to configure allowed origins
- Rate limiting middleware: token bucket vs sliding window concept
- Error handling middleware: catch unhandled errors, return consistent format
- Execution order matters — middleware runs in sequence

💡 Key phrase to own: 'Middleware handles cross-cutting concerns without cluttering route handlers.' Then name one you'd write for each project.

5. Dependency Injection ★ HIGH PRIORITY

- What DI solves: loose coupling, testability, avoiding global state
- Constructor injection vs function injection
- FastAPI's Depends() — how it wires DB sessions, auth, config into routes
- DI for testing: swap real DB with mock using the same interface
- IoC (Inversion of Control) container concept — brief vocabulary only

💡 FastAPI angle: 'Depends() let me inject the authenticated user object into any route without repeating auth logic — each route declared its dependencies and FastAPI resolved them.'

PHASE 3 — Data & Databases (Days 4–5)

6. SQL & Schema Design ★ HIGH PRIORITY

- SQL joins: INNER, LEFT, RIGHT — when to use each with example
- Indexes: what they are, when to add them, cost of over-indexing
- Normalization: 1NF (no repeating groups), 2NF (no partial dependency), 3NF (no transitive dependency)
- Foreign keys and referential integrity
- Transactions: ACID properties — Atomicity, Consistency, Isolation, Durability
- ORM vs raw SQL: tradeoffs (convenience vs performance vs control)
- Pydantic models as schema validation before DB writes

💡 Practice designing the DB schema for YOUR Social Spending Platform out loud: users table, expenses table (FK to users), categories table. Simple, real, yours.

7. Data Pipelines & ETL ♦ MEDIUM

- ETL: Extract, Transform, Load — what each stage does
- Batch vs real-time (streaming) ingestion — tradeoffs
- Data validation and cleaning in the pipeline

- How your House Price Predictor pipeline works end-to-end
- Serving ML model via REST API: request → pipeline → model → response

💡 *Walk through your House Price Predictor: 'Input comes via REST API, gets validated and transformed, the sklearn model runs inference, result returns in the response.' That IS a data pipeline.*

8. NoSQL Basics ♦ MEDIUM

- When to use NoSQL over SQL: flexible schema, horizontal scale, document data
- MongoDB document model: collections and documents vs tables and rows
- Key-value stores: Redis — caching, session storage, rate limiting
- CAP theorem: Consistency, Availability, Partition Tolerance — pick 2

💡 *You don't need deep NoSQL experience. Know WHEN you'd choose it and say: 'For the certificate tracking system, a relational schema was the right choice because of the structured, auditable nature of government records.'*

PHASE 4 — OOP & Code Design (Day 5–6)

9. OOP Principles ★ HIGH PRIORITY

- 4 pillars: Encapsulation, Abstraction, Inheritance, Polymorphism — one sentence + example each
- SOLID — S: Single Responsibility, O: Open/Closed, D: Dependency Inversion (most asked)
- Composition over inheritance — when and why
- Abstract classes vs interfaces — when to use each
- Design patterns vocabulary: Singleton, Factory, Strategy (brief — don't over-prepare)

💡 *Prepare one concrete example from your code: 'In the Social Spending Platform, each expense calculation strategy was a separate class implementing a common interface — composition over inheritance let me add new types without modifying existing code.'*

10. Error Handling & Logging ♦ MEDIUM

- Custom exception classes vs generic exceptions
- Global error handlers vs local try/catch — when to use each
- Structured logging: log levels (DEBUG, INFO, WARNING, ERROR, CRITICAL)
- What to log: request ID, timestamp, user ID, error message, stack trace
- Never log passwords, tokens, or PII

💡 *Say: 'I treat error handling as a first-class concern — I'd have a global middleware catch unhandled exceptions and return a consistent JSON error with a request ID for traceability.'*

PHASE 5 — Frontend Basics (Day 6)

Know enough to discuss, not to build. Be honest about your depth — interviewers respect it.

11. JavaScript & Async ♦ MEDIUM

- Promises: then/catch chaining
- async/await — syntactic sugar over Promises
- Fetch API pattern: fetch → await response → await .json()
- Event loop: why JS is single-threaded but non-blocking

- DOM: what it is, basic manipulation (querySelector, addEventListener)

🔗 Read [javascript.info async chapter](#) — 30 mins. You don't need to code JS; you need to discuss it. Frame it: 'My strength is backend; I've integrated JS frontends with my APIs.'

12. React & Frontend Architecture ♦ MEDIUM

- What React solves: component model, declarative UI, virtual DOM
- Component: props (input) vs state (internal) vs side effects (useEffect)
- SPA vs SSR: what each is and when you'd choose
- State management concept: local state vs global (Redux/Context) — vocabulary only
- How frontend connects to your FastAPI backend: fetch → JWT in header → parse response

PHASE 6 — System Design Vocabulary (Day 7)

Light prep. You need to participate in design discussions, not lead them from scratch.

13. Architecture Patterns ♦ MEDIUM

- Monolith vs microservices: when each makes sense, tradeoffs
- API gateway pattern: single entry point for all services
- Load balancer: distributes traffic across multiple server instances
- Horizontal scaling (more servers) vs vertical scaling (bigger server)
- Stateless vs stateful services — why stateless scales better

🔗 Be honest about microservices on your resume: 'I've designed modular services; I understand the distributed systems concepts, but haven't deployed at true microservices scale yet.'

14. Caching & Performance ♦ MEDIUM

- What caching solves: reduce DB load, speed up repeated reads
- Redis as a cache: set key → get key → TTL (time to live)
- Cache invalidation strategies: TTL, write-through, cache-aside
- When NOT to cache: data that changes often, user-specific data
- CDN: static assets cached at edge servers close to users

15. Message Queues & Async Processing ♦ GOOD TO KNOW

- What queues solve: decouple producer from consumer, handle spikes
- Producer → Queue → Consumer pattern
- Kafka (high-throughput streaming) vs RabbitMQ (task queues) — vocabulary only
- Use case: send email asynchronously without blocking the API response

🔗 Watch one Gaurav Sen system design video (Pastebin or URL shortener). 20 mins. You'll absorb the vocabulary needed to participate in any system design discussion.

16. Cloud & Deployment Basics ♦ GOOD TO KNOW

- What Docker does: containerize app with its dependencies
- What CI/CD is: automated build → test → deploy pipeline
- AWS vocabulary: EC2 (compute), S3 (storage), RDS (managed DB), Lambda (serverless)

- Environment variables: how to manage secrets/config across environments
- HTTP vs HTTPS: TLS handshake concept, why it matters

HOW TO USE THIS GUIDE

Method 1 — Teach-back (for concepts)

- Read a topic once. Close the tab. Explain it aloud as if teaching a classmate.
- Where you stumble = what to re-read. Repeat until fluent.
- Use Claude: ask it to explain a concept, then explain it back and ask for critique.

Method 2 — Anchor to your project (for answers)

- For every concept, prepare one sentence linking it to your project.
- Middleware → 'In my FastAPI app, I handled JWT auth in a middleware so route handlers stayed clean.'
- Theory without evidence is forgettable. Evidence without theory is shallow. Combine both.

Method 3 — Draw, don't code

- Draw the request lifecycle with middleware on paper.
- Draw JWT flow: client → request → middleware → verify → route → response.
- Draw your Social Spending Platform architecture. Practice walking through it out loud.

Method 4 — Mock interviews out loud

- Answer interview questions spoken aloud, not in your head.
- Time your project pitches — aim for 90 seconds per project.
- Practice: 'I don't know exactly, but I'd approach it by...' — shows thinking, not just recall.

Remember: *You won Smart India Hackathon. Interviewers are hiring someone who learns fast, reasons clearly, and owns their work. You have all three. Lead with that confidence.*